Part 3: Gradient Descent, Manual Calculations (matrix form)

Model. From the brief, $y = m_1x_1 + m_2x_2 + b$. In matrix form $\hat{y} = Xm + b$, i.e. $\hat{y}_i = m_1x_{i1} + m_2x_{i2} + b_i$, with n = 2 data points and b a vector with one entry per data point ($b = [1, 1]^T$).

$X = [[1, 3], [4, 10]]$, $y = [5, 6]^T$, $m_0 = [-1, 2]^T$, $b_0 = [1, 1]^T$, learning rate $\alpha = 0.01$ (not given in the assignment; chosen and stated).

Cost. $J(m,b) = (1/n)\ \Sigma\ (\hat{y}_i - y_i)^2 = (1/n)\ e^Te$, where $e = \hat{y} - y = Xm + b - y$.

Chain rule for $\partial J/\partial m$. $J = (1/n)\ e^Te \Rightarrow \partial J/\partial e = (2/n)\ e^T$. $e = Xm + b - y \Rightarrow \partial e/\partial m = X$. So $\partial J/\partial m = (\partial J/\partial e)(\partial e/\partial m) = (2/n)\ e^TX$, written as a column: $\partial J/\partial m = (2/n)\ X^Te$.

Chain rule for $\partial J/\partial b$. $\partial e/\partial b = I$ (identity, because b is a vector with one entry per data point). So $\partial J/\partial b = (\partial J/\partial e)(\partial e/\partial b) = (2/n)\ e^T\ I$, i.e. $\partial J/\partial b = (2/n)\ e$.

Same chain rule, element by element (no matrix shorthand). $J = (1/n)\ \Sigma_i\ (\hat{y}_i - y_i)^2$. $\partial J/\partial \hat{y}_i = (2/n)(\hat{y}_i - y_i) = (2/n)e_i$. $\partial \hat{y}_i/\partial m_j = x_{ij}$. $\partial \hat{y}_i/\partial b_k = 1$ if k = i, otherwise 0. Therefore $\partial J/\partial m_j = \Sigma_i\ (\partial J/\partial \hat{y}_i)(\partial \hat{y}_i/\partial m_j) = (2/n)\ \Sigma_i\ e_ix_{ij} = (2/n)(X^Te)_j$, and $\partial J/\partial b_k = (2/n)e_k$.

Update rule. $m \leftarrow m - \alpha\ \partial J/\partial m$, $b \leftarrow b - \alpha\ \partial J/\partial b$. Number of updates = number of group members = 3 (one per member).

Step-size check. The Hessian of J w.r.t. m is $(2/n)X^TX = [[17, 43], [43, 109]]$, largest eigenvalue ≈ 125.97, so α must be below $2/125.97 \approx 0.0159$. $\alpha = 0.01$ satisfies this.

```
ITERATION 1  (Member A)
Start: m = [-1, 2]^T,  b = [1, 1]^T
Step 1. Xm (row by row):
   row 1: (1)(-1) + (3)(2) = (-1) + (6) = 5
   row 2: (4)(-1) + (10)(2) = (-4) + (20) = 16
Step 2. y_hat = Xm + b:
   y_hat_1 = 5 + 1 = 6
   y_hat_2 = 16 + 1 = 17
Step 3. e = y_hat - y:
   e_1 = 6 - 5 = 1
   e_2 = 17 - 6 = 11
Step 4. J = (1/n) e^T e = (1/2)[(1)^2 + (11)^2] = (1/2)(1 + 121) = 61
Step 5. dJ/dm = (2/n) X^T e = (2/2) X^T e   [X^T uses COLUMNS of X]:
   component 1: (1)[(1)(1) + (4)(11)] = (1) + (44) = 45
   component 2: (1)[(3)(1) + (10)(11)] = (3) + (110) = 113
Step 6. dJ/db = (2/n) e = (2/2) e:
   component 1: (1)(1) = 1
   component 2: (1)(11) = 11
Step 7. Update with alpha = 0.01:   m_new = m - alpha * dJ/dm,   b_new = b - alpha * dJ/db
   m_1 = -1 - (0.01)(45) = -1 - (0.45) = -1.45
   m_2 = 2 - (0.01)(113) = 2 - (1.13) = 0.87
   b_1 = 1 - (0.01)(1) = 1 - (0.01) = 0.99
   b_2 = 1 - (0.01)(11) = 1 - (0.11) = 0.89
Check: new predictions = [2.15, 3.79],  new J = 6.5033

ITERATION 2  (Member B)
Start: m = [-1.45, 0.87]^T,  b = [0.99, 0.89]^T
Step 1. Xm (row by row):
   row 1: (1)(-1.45) + (3)(0.87) = (-1.45) + (2.61) = 1.16
   row 2: (4)(-1.45) + (10)(0.87) = (-5.8) + (8.7) = 2.9
Step 2. y_hat = Xm + b:
   y_hat_1 = 1.16 + 0.99 = 2.15
   y_hat_2 = 2.9 + 0.89 = 3.79
Step 3. e = y_hat - y:
   e_1 = 2.15 - 5 = -2.85
   e_2 = 3.79 - 6 = -2.21
Step 4. J = (1/n) e^T e = (1/2)[(-2.85)^2 + (-2.21)^2] = (1/2)(8.1225 + 4.8841) = 6.5033
Step 5. dJ/dm = (2/n) X^T e = (2/2) X^T e   [X^T uses COLUMNS of X]:
   component 1: (1)[(1)(-2.85) + (4)(-2.21)] = (-2.85) + (-8.84) = -11.69
   component 2: (1)[(3)(-2.85) + (10)(-2.21)] = (-8.55) + (-22.1) = -30.65
Step 6. dJ/db = (2/n) e = (2/2) e:
   component 1: (1)(-2.85) = -2.85
   component 2: (1)(-2.21) = -2.21
Step 7. Update with alpha = 0.01:   m_new = m - alpha * dJ/dm,   b_new = b - alpha * dJ/db
   m_1 = -1.45 - (0.01)(-11.69) = -1.45 - (-0.1169) = -1.3331
   m_2 = 0.87 - (0.01)(-30.65) = 0.87 - (-0.3065) = 1.1765
   b_1 = 0.99 - (0.01)(-2.85) = 0.99 - (-0.0285) = 1.0185
   b_2 = 0.89 - (0.01)(-2.21) = 0.89 - (-0.0221) = 0.9121
Check: new predictions = [3.2149, 7.3447],  new J = 2.49740005

ITERATION 3  (Member C)
Start: m = [-1.3331, 1.1765]^T,  b = [1.0185, 0.9121]^T
Step 1. Xm (row by row):
   row 1: (1)(-1.3331) + (3)(1.1765) = (-1.3331) + (3.5295) = 2.1964
   row 2: (4)(-1.3331) + (10)(1.1765) = (-5.3324) + (11.765) = 6.4326
Step 2. y_hat = Xm + b:
   y_hat_1 = 2.1964 + 1.0185 = 3.2149
   y_hat_2 = 6.4326 + 0.9121 = 7.3447
Step 3. e = y_hat - y:
   e_1 = 3.2149 - 5 = -1.7851
   e_2 = 7.3447 - 6 = 1.3447
Step 4. J = (1/n) e^T e = (1/2)[(-1.7851)^2 + (1.3447)^2] = (1/2)(3.18658201 + 1.80821809) = 2.49740005
Step 5. dJ/dm = (2/n) X^T e = (2/2) X^T e   [X^T uses COLUMNS of X]:
   component 1: (1)[(1)(-1.7851) + (4)(1.3447)] = (-1.7851) + (5.3788) = 3.5937
   component 2: (1)[(3)(-1.7851) + (10)(1.3447)] = (-5.3553) + (13.447) = 8.0917
Step 6. dJ/db = (2/n) e = (2/2) e:
   component 1: (1)(-1.7851) = -1.7851
   component 2: (1)(1.3447) = 1.3447
Step 7. Update with alpha = 0.01:   m_new = m - alpha * dJ/dm,   b_new = b - alpha * dJ/db
```

```
   m_1 = -1.3331 - (0.01)(3.5937) = -1.3331 - (0.035937) = -1.369037
   m_2 = 1.1765 - (0.01)(8.0917) = 1.1765 - (0.080917) = 1.095583
   b_1 = 1.0185 - (0.01)(-1.7851) = 1.0185 - (-0.017851) = 1.036351
   b_2 = 0.9121 - (0.01)(1.3447) = 0.9121 - (0.013447) = 0.898653
Check: new predictions = [2.954063, 6.378335],  new J = 2.164497790097
```

Summary of results

Iteration	m_1	m_2	b_1	b_2	J after update
0 (start)	-1	2	1	1	61
1	-1.45	0.87	0.99	0.89	6.5033
2	-1.3331	1.1765	1.0185	0.9121	2.49740005
3	-1.369037	1.095583	1.036351	0.898653	2.164497790097

Trend. J falls every update: 61 → 6.5033 → 2.4974 → 2.1645, so the updates reduce the error. The first step overshoots m (m_1: $-1 \to -1.45$, m_2: $2 \to 0.87$) because the first gradient (45, 113) is large; the next steps pull m back (m_2: $0.87 \to 1.1765 \to 1.0956$). The biases change little. The back-and-forth in m is caused by the steep direction of the cost surface, but $\alpha = 0.01$ is small enough for the error to keep decreasing.

Verification: these values equal the output of gradient_descent.py, and SciPy's numerical derivative of J matches the analytic gradient at the starting point ([45, 113] and [1, 11]).